

Devoir surveillé d'informatique

⚠ Consignes

- Les programmes demandés doivent être écrits en C ou en OCaml suivant l'exercice. Dans le cas du C, on suppose que les bibliothèques standards usuelles (`<stdio.h>`, `<stdlib.h>`, `<stdbool.h>`) sont déjà importées.
- On pourra toujours librement utiliser une fonction demandée à une question précédente même si cette question n'a pas été traitée.
- Veuillez à présenter vos idées et vos réponses partielles même si vous ne trouvez pas la solution complète à une question.
- La clarté et la lisibilité de la rédaction et des programmes sont des éléments de notation.

□ Exercice 1 : Analyse d'un algorithme

« Le problème du drapeau hollandais est un problème de programmation, présenté par Edsger Dijkstra, qui consiste à réorganiser une collection d'éléments identifiés par leur couleur, sachant que seules trois couleurs sont présentes (par exemple, rouge, blanc, bleu, dans le cas du drapeau des Pays-Bas). Étant donné un nombre quelconque de balles rouges, blanches et bleues alignées dans n'importe quel ordre, le problème est à les réarranger dans le bon ordre : les bleues d'abord, puis les blanches, puis les rouges. »

(Wikipedia)

On suppose déjà écrite la fonction `echange` de prototype `void echange(int tab[], int i, int j)` qui échange les éléments d'indice `i` et `j` dans le tableau `tab` et on considère dans la suite que cet échange s'effectue en temps constant. On donne ci-dessous une implémentation de l'algorithme du drapeau hollandais en langage C permettant de réarranger les éléments d'un tableau ne contenant les trois valeurs entières 1, 2 et 3 :

```
1 // Prend en entrée un tableau (ne contenant que les valeurs 1, 2 et 3)
2 // Ne renvoie rien
3 // Réarrange les éléments du tableau de façon à avoir les 1 puis les 2 et les 3
4 void drapeau_hollandais(int tab[], int taille)
5 {
6     int i1 = 0;
7     int i2 = taille - 1;
8     int i3 = taille - 1;
9     while (i1 <= i2){
10         if (tab[i1] == 1){
11             i1 = i1 + 1;
12         }
13         else{
14             if (tab[i1] == 2){
15                 echange(tab, i1, i2);
16                 i2 = i2 - 1;
17             }
18             else{
19                 echange(tab, i1, i2);
20                 echange(tab, i2, i3);
21                 i3 = i3 - 1;
22                 i2 = i2 - 1;
23             }
24         }
25     }
```

- Q1**– Faire fonctionner cet algorithme sur le tableau `tab = {1, 3, 2, 2, 3, 1}`, et donner le contenu de `tab` ainsi que celui des variables `i1`, `i2`, `i3` lors du déroulement de l'algorithme en recopiant et complétant le tableau suivant :

	<code>tab</code>	<code>i1</code>	<code>i2</code>	<code>i3</code>
Initialisation	{1, 3, 2, 2, 3, 1}	0	5	5
Etape 1	...	...	...	...
Etape 2	...	...	...	...
Etape 3	...	...	...	...
Etape 4	...	...	...	...
Etape 5	...	...	...	...
Etape 6	...	...	...	...

- Q2**– Prouver la terminaison de cet algorithme.

- Q3**– Prouver la correction de cet algorithme.

☞ *Indication : on pourra noter n la taille du tableau et :*

- P_1 la portion du tableau `tab` comprise entre les indices 0 (inclus) et `i1` (exclu)
- P_2 la portion du tableau `tab` comprise entre les indices `i2` (inclus) et `i3` (exclu)
- P_3 la portion du tableau `tab` comprise entre les indices `i3` (inclus) et $n - 1$ (exclu)

Et prouver l'invariant suivant : « P_1 ne contient que des 1, P_2 que des 2 et P_3 que des 3 ».

- Q4**– Donner, en la justifiant brièvement, la complexité de l'algorithme du drapeau hollandais.

- Q5**– On rappelle que l'algorithme du tri par sélection consiste à rechercher pour $i=0 \dots i=n-1$ le minimum du sous tableau `tab[i] ... tab[n-1]` et à l'échanger avec l'élément d'indice `i`. Donner une implémentation de cet algorithme de tri en langage C sous la forme d'une fonction de signature :

`void tri_selection(int tab[], int size)` qui prend en argument un tableau d'entiers et sa longueur et trie en place ce tableau. On pourra éventuellement utiliser une fonction annexe de recherche de minimum dont on précisera la spécification.

- Q6**– Rappeler, en la justifiant brièvement, la complexité de l'algorithme du tri par sélection.

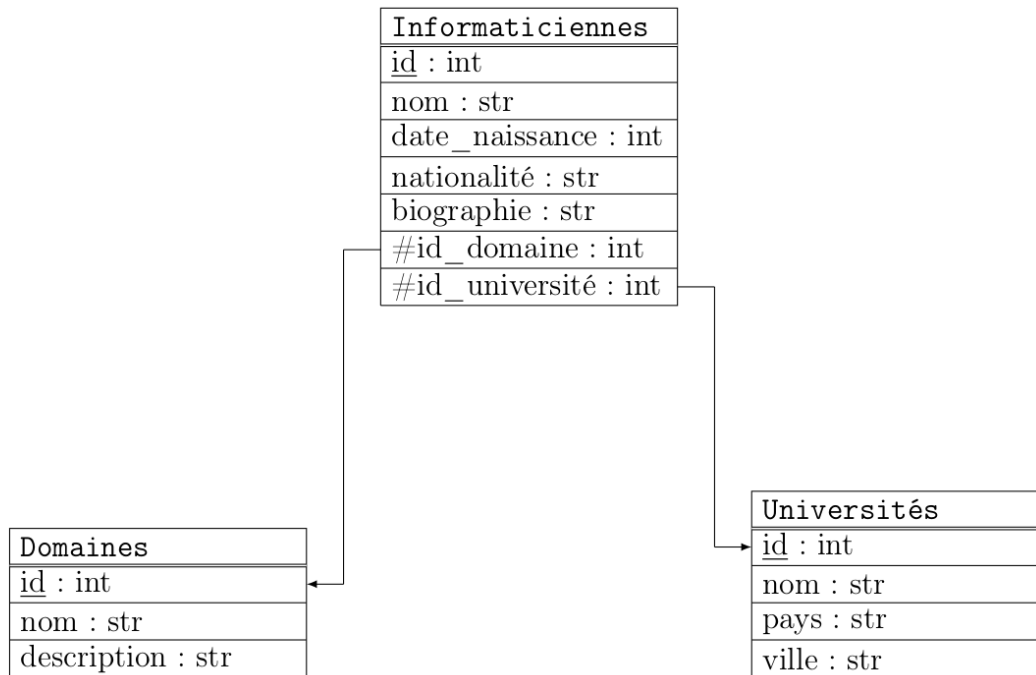
- Q7**– On a mesuré qu'en utilisant l'algorithme du tri par sélection un ordinateur trie une liste de dix million d'éléments en 5 secondes. Quel est le temps prévisible approximatif pour trier une liste contenant un milliard d'éléments ?

- Q8**– On a mesuré qu'en utilisant l'algorithme du drapeau hollandais un ordinateur trie une liste de dix million d'éléments en 0.2 secondes. Quel est le temps prévisible approximatif pour trier une liste contenant un milliard d'éléments ?

□ **Exercice 2** : Base de données et informaticiennes célèbres

🎓 CAPES L3 – Sujet zéro

On souhaite modéliser une base de données sur des informaticiennes ayant marqué l'histoire de l'informatique (exemples : Ada Lovelace, Grace Hopper, Frances Allen, Radia Perlman, ...). Cette base de données repose sur le schéma relationnel suivant



La table **Domaines** contient les différents domaines de recherche des universités.

- Q9**– Indiquer pour chaque table, la clé primaire et la ou les éventuelles clés étrangères.
- Q10**– Ecrire une requête SQL permettant d'afficher dans l'ordre alphabétique le nom des informaticiennes françaises.
- Q11**– Ecrire une requête SQL permettant d'afficher le nombre d'informaticiennes américaines.
- Q12**– Ecrire une requête SQL affichant le nom de chaque informaticienne, le nom de son domaine de recherche et le nom de son université.
- Q13**– Ecrire une requête SQL affichant le nombre d'informaticiennes par nationalité en triant le résultat par nombre décroissant.

□ Exercice 3 : L'accès aux enfers

Dans la mythologie grecque, l'accès aux Enfers est gardé par le Cerbère, un terrible loup à trois têtes. Celui-ci se trouve devant trois couloirs qui, soit permettent de rejoindre le monde des vivants, soit conduisent directement aux Enfers.

Lorsque Cerbère accueille un nouvel arrivant, il est contraint de lui dire la vérité. Par la suite, il peut mentir ou dire la vérité à sa guise mais il respecte toujours les règles qu'il s'est fixées.

Après avoir bu la coupe de ciguë, Socrate se retrouve face à Cerbère. Celui-ci, honoré de rencontrer le grand philosophe, veut lui offrir une chance d'éviter la damnation éternelle. Il lui dit alors : « *Je vais t'indiquer un des couloirs qui mène au monde des vivants mais, pour mettre à l'épreuve ta grande sagesse, j'énoncerai trois indications logiques qui seront, soit toutes vraies, soit toutes fausses et tu en déduiras le couloir que tu devras suivre* ».

Nous noterons I_1 , I_2 et I_3 les propositions associées aux indications de la première, la deuxième et la troisième tête de Cerbère.

- Q14**– Représenter l'énoncé de Cerbère sous la forme d'une formule du calcul des propositions dépendant de I_1 , I_2 et I_3

La première tête dit ensuite : « *Le premier couloir ainsi que le troisième mènent au monde des vivants* ». La deuxième tête dit : « *Si le deuxième couloir mène au monde des vivants, alors le troisième n'y mène pas* ». La troisième tête conclut par : « *Le premier couloir mène au monde des vivants, par contre le deuxième n'y mène pas* ».

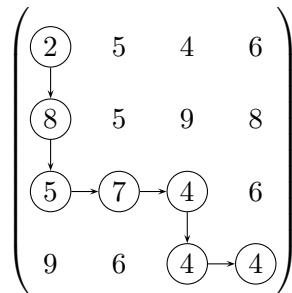
Nous noterons, C_1 , C_2 et C_3 les variables propositionnelles correspondant au fait que le premier, le deuxième et le troisième couloir mènent au monde des vivants.

- Q15**– Exprimer I_1 , I_2 et I_3 sous la forme du calcul des propositions dépendant de C_1 , C_2 et C_3

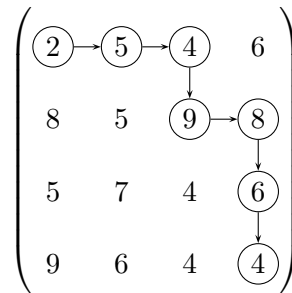
- Q16–** En utilisant le calcul des propositions (résolution avec les tables de vérité), déterminer le couloir que Socrate doit suivre pour rejoindre le monde des vivants.
- Q17–** En admettant que Cerbère ait menti en donnant les trois indications, Socrate pouvait-il suivre d'autres couloirs ? Si oui, lequel ou lesquels ?

□ **Exercice 4 :** *Traversée maximale d'une matrice carrée*

On considère une matrice à n lignes et n colonnes notée M à coefficients dans $\mathbb{N}$, et dont le coefficient situé à la ligne i et à la colonne j avec $0 \leq i \leq n-1$ et $0 \leq j \leq n-1$ sera noté $M_{i,j}$. On s'intéresse aux chemins depuis la première valeur en haut à gauche ($M_{0,0}$) jusqu'à la dernière en bas à droite ($M_{n-1,n-1}$) qui n'utilisent que les déplacements vers la droite ($\rightarrow$) ou vers le bas ($\downarrow$). Voici deux exemples de tels chemins dans une même matrice A de 4 lignes et 4 colonnes :



Un exemple C_1 de chemin dans A



Un exemple C_2 de chemin dans A

On appelle *somme d'un chemin* dans une matrice M , la somme des coefficients $M_{i,j}$ rencontrés en suivant ce chemin. Dans les exemples ci-dessus, le chemin C_1 a une somme de 34 et le chemin C_2 a une somme de 38. Le but du problème est d'étudier divers algorithmes qui cherchent à déterminer un chemin dont la somme est maximale.

On propose l'algorithme $\mathcal{A}_0$ suivant afin de résoudre ce problème : si les deux directions bas ou droite sont possibles on choisit celle qui mène vers le coefficient de matrice le plus élevé, en cas d'égalité, on choisit (arbitrairement) la direction bas. Dans l'exemple ci-dessus à partir de $M_{0,0} = 2$ on peut aller vers à droite vers $M_{0,1} = 5$ ou en bas vers $M_{1,0} = 8$ et on choisit donc d'aller en bas puisque $8 > 5$. Une fois arrivé en $M_{1,0}$ les deux directions possibles mènent vers des coefficients identiques (5 des deux côtés) et donc, on va vers le bas. Enfin, lorsqu'une seule direction est possible (parce qu'on a atteint la dernière colonne ou la dernière ligne), c'est cette direction qui est choisie.

- Q18–** Poursuivre le déroulement de cet algorithme sur la matrice A donnée en exemple ci-dessus, dessiner le chemin obtenu et donner sa somme.
- Q19–** Dans quelle famille d'algorithme peut-on classer cet algorithme ? Justifier.
- Q20–** Donner la complexité de cet algorithme.
- Q21–** Montrer en exhibant un exemple dans le cas $n = 4$ que cet algorithme ne fournit pas toujours un chemin de somme maximale.
- En Ocaml, on décide de représenter un déplacement vers la droite par un 0 et un déplacement vers le bas par un 1. Un chemin dans la matrice est alors représenté par une liste de 0 et de 1, par exemple le chemin C_1 donné en exemple est représenté par $[1, 1, 0, 0, 1, 0]$.
- Q22–** Ecrire une fonction `choix : int array array -> int -> int -> int` qui prend en argument une matrice ainsi qu'un numéro de ligne et de colonne et valide (c'est à dire positif et strictement inférieur à la dimension de la matrice) et renvoie le déplacement choisi (0 pour la droite et 1 pour le bas) par l'algorithme naïf $\mathcal{A}_0$.
- Q23–** Ecrire une fonction `naif : int array array -> int list` qui prend en entrée une matrice d'entiers M et renvoie le chemin fourni par l'algorithme naïf $\mathcal{A}_0$. Les réponses n'utilisant pas les aspects impératifs du langage (boucles et références) seront valorisées.
- Q24–** Ecrire une fonction `somme : int array array -> int list -> int` qui prend en entrée une matrice d'entiers M , un chemin dans cette matrice et renvoie la somme de ce chemin. De même que pour la question précédente, les réponses n'utilisant pas les aspects impératifs du langage seront valorisées.
- Q25–** Montrer que pour une matrice de taille n , un chemin de $M_{0,0}$ à $M_{n-1,n-1}$ est la donnée de $2n - 2$ déplacements et que parmi ces déplacements $n - 1$ sont vers la droite et $n - 1$ sont vers le bas.

- Q26–** En déduire que pour une matrice M de n lignes et n colonnes, il existe $\binom{2n-2}{n-1}$ chemins possibles de $M_{0,0}$ vers $M_{n-1,n-1}$.
- Q27–** En déduire la complexité de l'algorithme par force brute qui testerait le coût de tous les chemins possibles afin de déterminer le chemin optimal. On pourra utiliser le résultat suivant : $\binom{2n-2}{n-1}$ est un $\mathcal{O}(4^n)$.

On veut maintenant utiliser une méthode par programmation dynamique pour résoudre ce problème. On note, pour tout $i \in \llbracket 0; n-1 \rrbracket$ et $j \in \llbracket 0; n-1 \rrbracket$ $S_{i,j}$ la somme maximale des coefficients rencontrés depuis $M_{i,j}$ (exclu) jusqu'à $M_{n-1,n-1}$ (inclus).

- Q28–** Justifier rapidement que $S_{n-1,n-1} = 0$.
- Q29–** Justifier que pour tout $0 \leq i < n-1$, $S_{i,n-1} = M_{i+1,n-1} + S_{i+1,n-1}$ et que pour tout $0 \leq j < n-1$, $S_{n-1,j} = M_{n-1,j+1} + S_{n-1,j+1}$.
- Q30–** Montrer que pour $0 \leq i < n-1$ et $0 \leq j < n-1$, $S_{i,j} = \max \{M_{i+1,j} + S_{i+1,j}, M_{i,j+1} + S_{i,j+1}\}$.
- Q31–** Ecrire une fonction *réursive, dynamique* : `int array array -> int` qui calcule la solution du problème (à savoir $S_{0,0}$) en utilisant les relations de récurrence écrites aux questions précédentes.
- Q32–** Faire un schéma des premiers appels récursifs de la fonction précédente, quel phénomène constate-t-on ?
- Q33–** La mémoïsation est une méthode permettant de pallier le problème mis en évidence à la question précédente, expliquer brièvement en quoi cela consiste (on ne demande pas de programmer cette méthode mais simplement d'en expliciter le principe).
- Q34–** Déterminer la complexité de la méthode par programmation dynamique si on suppose qu'on utilise la mémoïsation.

□ Exercice 5 : Séquence génétique

On s'intéresse dans cet exercice à la compression de chaînes de caractères représentant des séquences génétiques codées sur l'alphabet $\{A, C, G, T\}$. Et on souhaite compresser la séquence $S=ATGTGATGTCCT$. On supposera qu'initialement la séquence est codée en ASCII et que donc chacun des caractères occupe 1 octet (donc 8 bits).

- Q35–** Construire l'arbre de Huffman associé à la compression de S .
- Q36–** Donner les codes obtenus pour chacun des quatre caractères A, C, G et T . En déduire le taux de compression de l'algorithme, sans tenir compte de la taille de l'arbre (on pourra conserver une écriture fractionnaire).

On rappelle que le principe de la compression LZW est d'attribuer un code aux préfixes rencontrés lors de la lecture du texte à compresser de façon à disposer d'un code compact si ce préfixe se présente à nouveau. On attribue initialement les codes $A \rightarrow 0$, $C \rightarrow 1$, $G \rightarrow 2$ et $T \rightarrow 3$. Le début de l'algorithme va donc consister à attribuer un nouveau code au premier préfixe non encore codé qui apparaît lors de la lecture du texte. Et donc, ici, on attribue le code 4 au préfixe AT et on émettra le code de A c'est à dire 0.

- Q37–** Poursuivre le déroulement de cet algorithme en complétant le tableau suivant :

Position dans le texte	Code émis	Nouveau préfixe ajouté
<u>A</u> TGTGATGTCCT	0	AT $\rightarrow$ 4

- Q38–** Quel est le taux de compression obtenu (la taille d'un code est un octet) ?
- Q39–** Décompresser le texte T codé par suite de codes $[3; 1; 4; 6; 5; 2; 0; 2]$ sur ce même alphabet en expliquant comment est reconstruit le dictionnaire de décompression.

A présent, on code en base 2 sur deux bits les nucléotides :

Nucléotide	A	C	G	T
Code sur 2 bits	00	01	10	11

Et on propose de représenter un brin d'ADN par une suite d'entiers codés sur 8 bits, les deux premiers bits de l'entier représentent le nucléotide et les 6 bits suivants indiquent le nombre de répétitions. Par exemple

- l'entier 10 s'écrit en binaire sur un octet $(00001010)_2$, les deux premiers bits sont 00 et donc représentent le nucléotide A, les 6 bits suivants donnent le nombre de répétitions $(001010)_2 = (10)_{10}$ et donc cet entier code AAAAAAAAAA.
- l'entier 167 s'écrit en binaire sur un octet $(10100111)_2$, comme 10 (les deux premiers bits) code le nucléotide G et que les 6 bits suivants représentent $(100111)_2 = (39)_{10}$ en décimal, cet entier représente le nucléotide G répété 39 fois.
- l'entier 200 s'écrit en binaire sur un octet $(11001000)_2$ et donc représente 8 répétitions du nucléotide T.

Q40– Que représente l'entier 101 ?

Q41– Par quel entier peut-on représenter TTTTTTTTTT (10 répétitions de T) ?

Q42– Avec ce codage, quel est le nombre maximal de répétitions d'un nucléotide que l'on peut coder avec un entier sur 8 bits ?